2.2.5 Using Rules to Define Functions

Section 1.6.1 discussed how you can define *functions* in *Mathematica*. In a typical case, you would type in `f[x_] = x^2` to define a function `f`. (Actually, the definitions in Section 1.6 used the `:=` operator, rather than the `=` one. Section 2.2.10 will explain exactly when to use `:=` and `=`.)

A definition like `f[x_] = x^2` can be thought of as a transformation rule. The definition specifies that whenever an expression of the form `f[`*anything*`]` occurs, it is to be replaced by *anything*`^2`.

This kind of "function definition" can be compared with the definitions like `f[a] = b` for "indexed variables" discussed in the last section. A definition like `f[a] = b` specifies that whenever the *particular* expression `f[a]` occurs, it is to be replaced by `b`. But the definition says nothing about expressions like `f[y]`, where `f` appears with another "index".

To define a "function", you need to specify values for expressions of the form `f[`*x*`]`, where the "argument" *x* can be *any* expression. This is what the `_` (pronounced "blank") in `f[x_]` means. The `_` stands for *any expression*. You can think of it a being a "blank space", where any expression could be inserted.

x also stands for any expression, but gives the expression the *name* *x*. Now you can understand how `f[x_] = x^2` specifies a transformation rule for all expressions of the form `f[`*anything*`]`. The *anything* is referred to as `x`, so the result *anything*`^2` is written as `x^2`.

`f[`*x*`] = `*value*	definition for a *specific expression* *x*
`f[`*x*`_] = `*value*	definition for *any expression*, referred to as *x*

The difference between defining an indexed variable and a function.

Making definitions for `f[2]` or `f[a]` can be thought of as being like giving values to various elements of an "array" named `f`. Making a definition for `f[x_]` is like giving a value for a set of "array elements" with arbitrary "indices". In fact, you can actually think of any function as being like an array with an arbitrarily-variable index.

In mathematical terms, you can think of `f` as a *mapping*. When you define values for, say, `f[1]` and `f[2]`, you specify the image of this mapping for various discrete points in its domain. Defining a value for `f[x_]` specifies the image of `f` on a continuum of points.

Since definitions for objects `f[1]`, `f[a]` and `f[x_]` are all just treated as trans-

formation rules, you can mix these definitions in any way.

This defines a transformation rule for the *specific expression* `f[x]`.

```
In[1]:= f[x] = u
Out[1]= u
```

When the specific expression `f[x]` appears, it is replaced by `u`. Other expressions of the form `f[`*argument*`]` are, however, not modified.

```
In[2]:= f[x] + f[y]
Out[2]= u + f[y]
```

This defines a value for `f` with *any expression* as an "argument".

```
In[3]:= f[x_] = x^2
                2
Out[3]= x
```

The old definition for the *specific expression* `f[x]` is still used, but the new general definition for `f[x_]` is now used to find a value for `f[y]`.

```
In[4]:= f[x] + f[y]
                  2
Out[4]= u + y
```

This removes all definitions for `f`.

```
In[5]:= Clear[f]
```

It is very convenient to define mathematical functions using transformation rules. An example is the factorial function. This function is in fact built in to *Mathematica* (it is written n`!`). However, as an illustration, we will show how you could define the function for yourself.

The mathematical definition of the factorial function is: $f(1) = 1$; $f(n) = n\ f(n-1)$. This mathematical definition can immediately be thought of in terms of transformation rules. For any n, $f(n)$ should be replaced by $n\ f(n-1)$, except that $f(1)$ should simply be replaced by 1.

You can enter the mathematical definition of factorial almost directly into *Mathematica*, in the form: `f[1] = 1;  f[n_] := n f[n-1]`.

The exact distinction between `=` and `:=` will be discussed in Section 2.2.10. For now, a good procedure is to use `:=` whenever there is a `_`, and `=` when there is not.

Here is the value of the factorial function with argument 1.

```
In[6]:= f[1] = 1
Out[6]= 1
```

Here is the general recursion relation for the factorial function.

```
In[7]:= f[n_] := n f[n-1]
```

Now you can use these definitions to find values for the factorial function.

```
In[8]:= f[10]
Out[8]= 3628800
```

The results are the same as you get from the built-in version of factorial.

```
In[9]:= 10!
Out[9]= 3628800
```

This shows the definitions made for your factorial function f. Notice that the definition for the specific expression f[1] comes before the general definition for f[n_].

```
In[10]:= ?f

f
f/: f[1] = 1
f/: f[n_] := n f[n - 1]
```

The example of the factorial function illustrates several important points.

When you enter the expression f[10], *Mathematica* follows its general principle of applying transformation rules until no more rules apply. Its first step is to apply the rule f[n_] := n f[n-1] to get the result 10 f[9]. Then it applies this rule eight more times to the succession of expressions f[9], f[8], etc. Finally, the expression f[1] is generated. Now *Mathematica* applies the rule f[1] = 1. Collecting all the intermediate terms together, it gets the final result 3628800.

An important point is that *Mathematica* tries to apply the specific rule for f[1] *before* it applies the general rule for f[n_].

This is an example of a principle that *Mathematica* follows. When you give a sequence of transformation rules, *Mathematica* arranges them so that the *most specific rules come first*. Then, when it tries to apply the rules, it tests them in order. If a specific rule does apply, then it is used; only if none of the specific rules apply will the later, more general, rules be used. If *Mathematica* cannot decide whether a new rule is more or less specific than the ones it already knows, it puts the new rule at the end of the list.

For the factorial function, the "end condition" f[1] = 1 is a specific rule, which is placed before the general rule f[n_] := n f[n-1]. The order in which *Mathematica* puts the rules does not depend on the order in which you typed the definitions for f[1] and f[n_].

Many mathematical functions can conveniently be defined by giving a few special cases, and then a general formula. Often, as for the factorial function, the general formula will be *recursive*, in the sense that it defines one instance of the function in terms of other instances of the same function.

Whenever you have a recursive formula, you must be careful to make sure that you specify all the necessary "end conditions". If you forgot to give the value of f[1] for the factorial function, then evaluating f[10] would lead to an infinite sequence of f's, with successively more negative arguments. If *Mathematica* gets into this kind of infinite loop, you can always stop it by making an interrupt as discussed in Section 1.3.6. You can then type in the end conditions, and try the computation

again.